IHK Deckblatt

Synchronisation externer Veranstaltungsdaten mit WordPress & Modern Events Calendar

ABSCHLUSSPROJEKT

PRÜFLING

30. OKTOBER 2025

Projektdokumentation

Fachinformatiker Anwendungsentwicklung

Identnummer:

Autor:

Prüfungsstelle

IHK Halle-Dessau, Franckestraße 5, 06110 Halle (Saale)

© Verfasser
© X (Nutzungsrechtinhaber)

Diese Dokumentation wurde im Rahmen der Abschlussprüfung zum Fachinformatiker – Anwendungsentwicklung bei der X erstellt.

Alle Urheberrechte verbleiben beim Verfasser.
Die X besitzt ein einfaches Nutzungsrecht an den im Rahmen des Projekts entstandenen Inhalten.
Die Verwendung, Vervielfältigung oder Veröffentlichung dieses Dokuments ist nur mit ausdrücklicher Genehmigung des Verfassers und der X gestattet.

Sitz der Gesellschaft:
UST-ID:
Amtsgericht
Geschäftsführer:

1. Inhaltsverzeichnis

1. Inhaltsverzeichnis i
1.1. Tabellenverzeichnis iii
1.2. Abbildungsverzeichnis iii
1.3. Listings iii
2. Dokumentation iv
2.1. Information 1
2.1.1. Arbeitsumfeld 1
2.1.2. Klärung der externen Datenquelle/Formate 1
2.1.3. Ist-Analyse 1
2.1.4. Soll-Konzept und Rahmenbedingungen 1
2.1.5. Analyse der externen und internen APIs 2
2.1.6. Risikobewertung 2
2.1.7. Wirtschaftlichkeitsanalyse 2
2.2. Planung 3
2.2.1. Arbeitsschritte und Zeitplan anpassen 3
2.2.2. Alternativen und Machbarkeit 3
2.2.3. Arbeitsmittel 3
2.2.4. Erstellung des Pflichtenhefts 4
2.2.5. Entwurf der Plugin Struktur und des Lösungskonzepts 4
2.2.6. Entwurf des Datenbankmodells 5
2.2.7. Entwurf der Admin-Oberflächen 5
2.3. Durchführung 5
2.3.1. Datenbank Konzept umsetzen 5
2.3.2. Entwicklung der Admin-Oberflächen 6
2.3.3. Entwicklung des Import Moduls 6
2.3.4. Entwicklung des Export Moduls 7
2.3.5. Konfiguration des Schedulers 7
2.4. Kontrolle und Abschluss 8
2.4.1. Durchführen von Unit-Tests 8
2.4.2. Funktionstest und Systemtests 8
2.4.3. Eingehaltene Sicherheitsaspekte überprüfen 9
2.4.4. Soll-Ist-Vergleich und Ausblick 9
2.4.5. Benutzungsanleitung und Verlaufsdokumentation 10
2.4.6. Abnahme und Lesson learned 10
2.4.7. Zeitkontrolle 10

3. Anlagen ____ i
Anlage 1 Auszug aus dem Pflichtenheft - Anforderungen ____ ii
Anlage 2 JSON-Beispiel der externen API ____ iv
Anlage 3 Risikoanalyse - Auszug aus dem Pflichtenheft ____ v
Anlage 4 Wirtschaftlichkeitsberechnung ____ vi
Anlage 5 Zeitplan Soll-Ist-Vergleich ____ vi
Anlage 6 Komponentendiagramm ____ vii
Anlage 7 Klassendiagramm ____ viii
Anlage 8 Datenbanktabellen – ER-Modell ____ ix
Anlage 9 Wireframes und GUIs ____ x
Anlage 10 Sequenzdiagramm Importer ____ xi
Anlage 11 Code Auszug Importer und ImporterFactory Klassen ____ xii
Anlage 12 Sequenzdiagramm Exporter ____ xiii
Anlage 13 Code Auszug EventExportController Klasse ____ xiv
Anlage 14 Server-Cronjob ____ xv
Anlage 15 Auszug Unit Tests ____ xv
Anlage 16 Auszug von Testfällen ____ xvi
Anlage 17 Ergebnis importierter Events ____ xvii
Anlage 18 Rest-API Zugriff ____ xvii
Anlage 19 Auszug Benutzerhandbuch ____ xviii
Anlage 20 Auszug Verlaufsdokumentation ____ xviii
Anlage 21 Glossar ____ xix
4. Quellen ____ xix

1.1. Tabellenverzeichnis

Anlage 3 Risikoanalyse - Auszug aus dem Pflichtenheft v
Anlage 4 Wirtschaftlichkeitsberechnung vi
Anlage 5 Zeitplan Soll-Ist-Vergleich vi
Anlage 16 Auszug von Testfällen xvi
Anlage 21 Glossar xix

1.2. Abbildungsverzeichnis

Anlage 2 JSON-Beispiel der externen API iv
Anlage 6 Komponentendiagramm vii
Anlage 7 Klassendiagramm viii
Anlage 8 Datenbanktabellen – ER-Modell ix
Anlage 9 Wireframes und GUIs x
Anlage 10 Sequenzdiagramm Importer xi
Anlage 11 Code Auszug Importer und ImporterFactory Klassen xii
Anlage 12 Sequenzdiagramm Exporter xiii
Anlage 13 Code Auszug EventExportController Klasse xiv
Anlage 14 Server-Cronjob xv
Anlage 15 Auszug Unit Tests xv
Anlage 17 Ergebnis importierter Events xvii
Anlage 18 Rest-API Zugriff xvii

1.3. Listings

Anlage 1 Auszug aus dem Pflichtenheft - Anforderungen ii
Anlage 19 Auszug Benutzerhandbuch xviii
Anlage 20 Auszug Verlaufsdokumentation xviii
4. Quellen xix

2. Dokumentation

2.1. Information

2.1.1. Arbeitsumfeld

X ist eine Digitalagentur aus - , die Unternehmen in den Bereichen Online-Marketing, Webentwicklung, IT-Infrastruktur und Digitalisierung unterstützt. Im Auftrag eines Kundenprojekts wurde ein WordPress-Plugin realisiert, das Veranstaltungsdaten automatisch aus einer externen Quelle in das Plugin Modern Events Calendar (MEC) integriert. Die Entwicklung erfolgte vollständig aus dem Homeoffice, wobei das Wasserfallmodell als Vorgehensmodell zugrunde lag. Dies basierte auf dem Umfang und der bestehenden Anforderungen durch das Lastenheft des Projekts.

2.1.2. Klärung der externen Datenquelle/Formate

Mit dem Kunden und Projektbetreuer musste vorab noch abgestimmt werden, welche Schnittstelle und welches Format verwendet werden sollen. Die Eventdaten werden über eine REST-API im JSON Format bereitgestellt, wofür die Schnittstellendokumente angefordert wurden.

2.1.3. Ist-Analyse

Im Rahmen des Kundenauftrags wurden Eventdaten von externen Quellen bezogen. Zuvor kopierten Mitarbeiter die Eventdaten manuell von den jeweiligen Partner-Websites und trugen sie ins WordPress-Plugin Modern Events Calendar (MEC) ein. Dieser Prozess war zeitintensiv und fehleranfällig, da er durch die manuelle Eingabe zu Dubletten, Tippfehlern und Verzögerungen führte. Der Ablauf bestand darin, dass Mitarbeiter die Webseiten der Quellen aufgerufen und relevante Eventinformationen kopiert sowie einzeln im MEC-Plugin eingetragen hatten. Dies verursachte einen hohen Aufwand, der mit zunehmender Anzahl an Daten und Events weiter angestiegen wäre, was die Skalierbarkeit stark eingeschränkt hätte. Es existieren APIs, die genutzt werden können, um den Prozess zu automatisieren. Die bisher eingesetzten Tools umfassen WordPress und das MEC-Plugin, die jedoch keine automatische Datenübernahme ermöglichen. Insgesamt zeigte die Analyse, dass die manuelle Pflege ineffizient war und eine Automatisierung sinnvoll war, um Zeit zu sparen, Fehler zu vermeiden und die Datenqualität zu verbessern.

2.1.4. Soll-Konzept und Rahmenbedingungen

Ziel des Projekts war die Entwicklung eines WordPress-Plugins, das automatisch Veranstaltungen aus einer externen Quelle in festgelegten Intervallen importiert.

Das Plugin sollte die importierten Veranstaltungen mit bestehenden Einträgen abgleichen, um Dubletten zu vermeiden. Zudem sollte eine Export-Schnittstelle für berechtigte Kunden mittels eines API-Keys bereitgestellt werden, damit veröffentlichte Events in andere Systeme exportiert und weiterverwendet werden können. Das Plugin sollte so konzipiert werden, dass es erweiterbar und wartbar bleibt, ohne Änderungen am MEC Core vorzunehmen. Die Implementierung sollte in einer eigenen Struktur erfolgen, die weder den MEC-Core noch den WordPress-Core verändert. Dies sollte sicherstellen, dass das Plugin flexibel und zukunftssicher bleibt. Durch die Automatisierung des Imports sollte der Pflegeaufwand erheblich reduziert und die Aktualität der Eventdaten gewährleistet sowie die Skalierbarkeit des Systems zukünftig sichergestellt werden. Die detaillierten Anforderungen und Spezifikationen sollten im Lasten- und Pflichtenheft beschrieben werden. Im WordPress-Backend sollte eine Adminoberfläche für die Verwaltung der Datenquellen sowie der Export-Kunden integriert werden. Über diese Adminoberfläche sollten auch der Import-Zeitplan und die Deaktivierung des Importprozesses gesteuert werden. Fehler und Importvorgänge sollten in der Datenbank und Logdatei protokolliert werden. Für die Anforderungen wird auf den Auszug aus dem Pflichtenheft auf die Anlage 1 verwiesen.

2.1.5. Analyse der externen und internen APIs

Nach der Analyse der JSON-Schnittstelle können die Daten über das HTTP-Protokoll abgerufen werden. Sie sind in mehreren Objekten strukturiert, darunter Event, Veranstalter, Datum und Zeit sowie Multimedia, das unter anderem Bilder und weitere Inhalte verlinkt oder speichert. Die Struktur wurde in Anlage 2 (*vom Kunden erhalten) hinzugefügt. Die Interne API ist in WordPress über das Plugin Modern Events Calendar realisiert. Hier werden bei der neuen Anlegung von Events die WordPress Datenbank genutzt und gepflegt, aus dieser die Events im Frontend dargestellt werden.

2.1.6. Risikobewertung

Zur Bewertung möglicher technischer Risiken und Veränderungen wurde eine Risikoanalyse durchgeführt. Die Ergebnisse sind in einer Risikotabelle in Anlage 3 zusammengefasst.

2.1.7. Wirtschaftlichkeitsanalyse

Das Projekt wurde im Rahmen der externen Prüfung ohne Vergütung durchgeführt. Daher fielen keine Personalkosten an. Es blieben die Kosten für eigene technische Ressourcen und Arbeitgeberkosten wie Server, Internet und Strom. Für diese Punkte wurde ein pauschaler Stundensatz von X Euro angesetzt.

Zusätzlich kamen Kosten für die Projektbegleitung auf. Hierfür wurde ein Stundensatz von X Euro berechnet. Dieser Stundensatz umfasst auch weitere Betriebskosten des Projekts bzw. der Firma als Arbeitgeberkosten. Weitere Kosten fielen nicht an, da kostenfreie Werkzeuge und Dienste verwendet wurden. In Anlage 4 befindet sich die Kostenberechnung. Die Amortisation erfolgt beim Kunden, da es sich bei dem Projekt um einen einmaligen Auftrag und Ertrag handelte.

2.2. Planung

2.2.1. Arbeitsschritte und Zeitplan anpassen

Der im Projektantrag grob erstellte Zeitplan wurde zu Anfang des Projekts überarbeitet, da dieser nicht optimal war und es einer zusätzlichen Aufteilung bedurfte, um einen besseren Ablauf zu gewährleisten. Die aktualisierte Planung ist in Anlage 5 zu finden. Das Projekt wurde vom 18. September bis zum 30. Oktober 2025 durchgeführt.

2.2.2. Alternativen und Machbarkeit

Es wurde frühzeitig überprüft, ob es bereits eine alternative Lösung zum Import der benötigten Daten gibt. Dazu wurde sich auf dem Markt, unter anderem über gängige Systeme wie Google, WordPress und ChatGPT umgeschaut, ob es Plugins oder Ansätze gibt, die eine Synchronisierung von Daten aus externen APIs über JSON oder andere Formate ermöglichen. Dabei zeigte sich, dass zwar verschiedene Alternativen existieren, diese jedoch das gewünschte Ziel nicht vollständig abdecken. Insbesondere war es erforderlich, speziell auf die jeweilige API-Struktur reagieren zu können, was von den bestehenden Lösungen nicht ausreichend unterstützt wird. Daher wurde schnell entschieden, eine eigene Lösung zu entwickeln, da dies in diesem Fall die flexibelste und sinnvollste Option war und sich aufgrund der vorhandenen Infrastruktur problemlos in WordPress umsetzen ließ.

2.2.3. Arbeitsmittel

Für die Implementierung wurde ein Klon der bestehenden WordPress-Instanz inklusive des MEC-Plugins sowie der zugehörigen Datenbank vom Liveserver bereitgestellt. Zusätzlich bestand die Möglichkeit, auf einer internen Serverumgebung zu entwickeln. Da WordPress PHP nutzt wurde für die Entwicklung PHP sowie gängige kostenfreie Werkzeuge wie Visual Studio Code, XAMPP, Docker und WireframeSketcher (WireframeSketcher, 2025) genutzt.

2.2.4. Erstellung des Pflichtenhefts

Nach der Analysephase konnte auf Grundlage des Lastenhefts das Pflichtenheft erstellt werden. Darin wurden die geplante Umsetzung sowie die zuvor definierten Ziele detaillierter beschrieben, um eine klare Grundlage und einen späteren Vergleich zu haben. Das Lastenheft wurde bereits vom Kunden mit dem Projektbegleiter erstellt und für das Projekt bereitgestellt. Um Redundanz in den Anlagen zu vermeiden, werden die Anforderungen aus dem Pflichtenheft in Anlage 1 dargestellt.

2.2.5. Entwurf der Plugin Struktur und des Lösungskonzepts

Hier begann die Planung der Struktur des Plugins sowie der Architektur, um künftig gegebenenfalls weitere Quellen und zusätzliche Module anbinden zu können. Im Rahmen des Plugins wurde eine modulare, schichtenbasierte Architektur innerhalb eines monolithischen Systems gewählt. Die Entscheidung hierfür ist naheliegend: Die Module können unabhängig voneinander arbeiten und spiegeln zugleich die funktionalen Anforderungen wider. Diese Trennung gewährleistet eine bessere Wartbarkeit, Erweiterbarkeit und Testbarkeit der Lösung, ohne auf typische WordPress-Mechanismen verzichten zu müssen. Die Hauptkomponenten setzen sich wie folgt zusammen: modulare Administrationsoberflächen zur Eingabe und Verwaltung der Datenquellen und Kunden, ein Importer-System, das auf einem gemeinsamen Interface basiert und die Erweiterung um neue Formate erlaubt, ein Exporter-System zur Ausgabe der Daten über die WordPress Rest-API sowie eine zentrale Datenbankklasse im Singleton-Pattern, die sämtliche Datenbankoperationen kapselt. Ziel war es, mit dem Factory- und Strategy-Pattern zu arbeiten, um die Erweiterbarkeit sicherzustellen. Das Factory Pattern sorgt dafür den entsprechenden Importer für das von der Quelle angegeben Datenformat an der richtigen Stelle auszuwählen. So können später problemlos weitere Importer wie XML, CSV oder auch Ical genutzt und implementiert werden. Das Strategy Pattern wird zusätzlich dazu genutzt, um den entsprechenden Parser für die Quelle auszuwählen. Da die Quellen unterschiedliche Strukturen der Daten anbieten, muss hierauf mit spezifischen Algorithmen reagiert werden. Daher ist das Strategy-Pattern dafür zuständig später auch weitere Quellstrukturen auszuwählen und mit dem richtigen Parser zu reagieren. Hierzu wurden ein Komponenten- und Klassendiagramm ausgearbeitet, diese in Anlage 6 Komponentendiagramm und in Anlage 7 Klassendiagramm enthalten sind.

2.2.6. Entwurf des Datenbankmodells

Die bestehende Datenbank wurde um vier weitere Tabellen erweitert. Die Tabelle *`wp_event_sources`* dient der Erfassung der Datenquellen und wird für den Importprozess benötigt, um die APIs zu beschreiben, welche angesprochen werden sollen. Die Tabelle `wp_export_clients` ist im Zusammenhang mit der REST-API relevant und ermöglicht die Bereitstellung von Kundendaten sowie von Exportfunktionen. Die Export Tabelle besteht unabhängig von anderen Tabellen. Ergänzend dazu werden die Tabellen `wp_event_sync_log` und `wp_event_sync_log_details` für die Protokollierung der Synchronisationsvorgänge verwendet, als auch um die Events zuzuordnen. Zusätzlich wurden auch die bestehenden Tabellen angepasst: In der Tabelle `wp_mec_events`, die für die Speicherung der Event-Daten verantwortlich ist, wurde das neue Feld `event_origin` ergänzt, um die Herkunft der Events nachvollziehbar zu machen. Die Tabelle `wp_posts`, und `wp_mec_dates` die für die Veröffentlichung und Verwaltung von Event-Posts, Inhalten und Daten zuständig sind (WordPress-Dokumentation, 2025), wurden inhaltlich nicht verändert, jedoch zur Veranschaulichung der Beziehungen im Entity-Relationship-Modell ebenfalls berücksichtigt, welches sich in Anlage 8 befindet.

2.2.7. Entwurf der Admin-Oberflächen

Für die Admin-Oberflächen wurde vorab mithilfe des WireframeSketcher (WireframeSketcher, 2025) ein Entwurf erstellt. Die Eingabefelder wurden basierend auf der definierten Tabellenstruktur gewählt und im Benutzerdokument beschrieben. Entsprechende Buttons wurden ebenfalls berücksichtigt. In Anlage 9 befinden sich die Wireframes sowie der Vergleich zum umgesetzten Design.

2.3. Durchführung

2.3.1. Datenbank Konzept umsetzen

Basierend auf dem ER-Modell wurden die notwendigen Tabellen in der WordPress-Datenbank erstellt. Dafür kamen SQL-CREATE-Befehle zum Einsatz, um die Tabellenstruktur in die Datenbank zu integrieren. Jede Tabelle wurde entsprechend der im Modell definierten Entitäten und deren Beziehungen aufgebaut. Dabei wurden die entsprechenden Spalten und Datentypen festgelegt und auch Verknüpfungen zwischen den Tabellen durch Primär- und Fremdschlüssel sichergestellt und getestet. Die Tabellen wurden lokal als auch auf dem Live-Server erstellt, um bereits bei der Programmierung die Funktionalität einsehen zu können. Zum erneuten Abgleich befinden sich die Tabellen in Anlage 8.

2.3.2. Entwicklung der Admin-Oberflächen

Das Plugin konnte lokal entwickelt werden. Durch die Erstellung der Tabellen konnte der Ablauf 1 zu 1 später auf dem Server übernommen werden da hier lediglich das Plugin eingebunden werden musste. Die Administrationsoberflächen wurden auf Grundlage der zuvor erstellten Wireframes umgesetzt. Zu diesem Zweck wurde die Haupt-Plugindatei erstellt und um zwei Subpages erweitert, die über WordPress eingebunden werden. Diese Hauptdatei muss sich im Hauptverzeichnis des Plugins befinden, um korrekt von WordPress erkannt und geladen zu werden (WordPress-Plugins, 2025). Für den Import und Export wurde jeweils eine eigenständige GUI mit PHP erstellt, die die erforderlichen Eingabefelder für datenbankspezifische Abfragen bereitstellt. Beide PHP-Dateien sind strukturell identisch, da sie eine vergleichbare Funktionalität bieten. Die Importer-Datei enthält jedoch zusätzliche Eingabefelder zur Verarbeitung der Quellinformationen. Im Sinne von Security by Design wurde durchgängig darauf geachtet, sämtliche Datenbankoperationen unter Verwendung von „prepared Statements“ umzusetzen, um potenziellen SQL-Injection-Angriffen oder Missbrauch, insbesondere bei einer Kompromittierung des WP-Adminbereichs, vorzubeugen. Die Datenbankinstanz wird über die Klasse DatabaseService verwaltet und als Singleton über die Methode `get_instance()` bereitgestellt. Bereits gespeicherte Quellen werden beim Laden aus der Datenbank abgerufen und in einer foreach-Schleife als HTML-Elemente ausgegeben. Dabei wird geprüft, ob der „active“ Status in der Datenbank auf „true“ gesetzt ist, um die jeweilige Quelle als editierbar anzuzeigen. Über den „+“ Button können zusätzliche Quellen und Clients hinzugefügt und über den „Daten aktualisieren“ Button an den Server übermittelt werden. Bestehende Einträge können über den „Eintrag löschen“-Button entfernt werden. Für die korrekte Darstellung der Felder und Funktionenwurden JavaScript und CSS verwendet. Das Ergebnis befindet sich in Anlage 9.

2.3.3. Entwicklung des Import Moduls

Anhand der Klassenstruktur und des Komponentendiagramms wurden die Klassen und Interfaces erstellt und implementiert. In der Importer-Klasse werden die Hauptfunktionen aufgerufen. Dabei wird über die Quellen iteriert, die aus der Datenbank abgerufen werden. Es wird geprüft, ob für eine Quelle und dem angegebenen Intervall ein Import ausgelöst werden soll. Nach dieser Prüfung erfolgt über die `ImporterFactory.create()` Methode die Auswahl des richtigen Importers für das jeweilige Format. Der Importer erhält durch die `createJsonParser()` Methode den passenden Parser und damit den richtigen Algorithmus für die Quelle zurück. Zudem wird eine Instanz der Datenbank über die Singleton-Methode `getInstance()` der Datenbankklasse übergeben, um damit weiterarbeiten zu können. Nach Rückgabe des passenden Parsers wird der Importer instanziiert und die

vordefinierten Funktionen des ImporterInterfaces können aufgerufen werden. Die `fetchData()` Methode ruft nun die Daten von den Quellen über die URL ab, sodass diese mit der PHP-Methode `json_decode()` strukturiert zurückgegeben werden. Folgend durch die Duplikat Prüfung `filterDuplicates()` werden die Event-IDs der Quelle in der Datenbank mit den neuen Daten verglichen und nicht benötigte Daten aussortiert. Anschließend werden die Daten über die `parseData()` Methode an die `transform()` Methode des Parsers weitergegeben, um sie nach einem bestimmten Algorithmus für die Datenbank aufzubereiten. So können die Tabellen mit den Events im nächsten Schritt über die `saveToDatabase()` Methode gefüllt werden. Wichtige Prozesse oder Fehler werden geloggt und in der Datenbank und Logdatei gespeichert. Dazu wurde der WordPress log in der *`wp-config`* aktiviert (WordPress, 2025). Als Ablauf und Dokumentation wurde hierfür ein Sequenzdiagramm erstellt und in Anlage 10 Sequenzdiagramm Importer hinzugefügt sowie die Importer Klassen aus dem Code in Anlage 11 Code Auszug Importer und ImporterFactory Klassen gezeigt.

2.3.4. Entwicklung des Export Moduls

Für die Export-REST-API mussten die Routen in der Haupt-Plugindatei gesetzt werden. Um die Sicherheit und Authentifizierung zu gewährleisten, mussten der Callback und der Permission-Callback angegeben werden. Dazu wurde in der Haupt-Plugindatei event-source-Manager eine Referenz auf die Klasse EventExportController erstellt und deren Methoden `validateApiKey()` und `handleExportRequest()`. Diese Methoden sind dafür verantwortlich, die Exporte zu erstellen. Dabei wird zunächst immer der API-Key überprüft. Sollte dieser vorhanden und korrekt sein, erfolgt die Weiterleitung an die `handleExport()` Funktion. Diese lädt für den jeweiligen Client die Daten im JSON-Format und gibt sie zurück. Als Filter kann auch in der Anfrage, der Zeitraum und Kategorie angegeben werden. Der API-Key wird aktuell in der URL übermittelt. Hierzu befinden sich ein Sequenzdiagramm und Codeausschnitt in Anlage 12 Sequenzdiagramm Exporter und Anlage 13 Code Auszug EventExportController Klasse.

2.3.5. Konfiguration des Schedulers

Schließlich wurde der Scheduler für die Implementierung auf dem Server eingerichtet. Hierbei fiel die Wahl auf einen Server-Cron, um keine zusätzliche Abhängigkeit von WordPress zu erzeugen. Bei einem möglichen WordPress-Cron müsste nämlich auf den Aufruf der Website gewartet werden, da der WP-Cron nur dann ausgeführt wird, wenn die Seite besucht/geladen wird (WordPress-Dokumentation, 2025). Hierzu wurde mit dem

Standarduser, dieser die Rechte besitzt, `crontab -u www-data -e` ausgeführt und eingerichtet. Der Scheduler wurde lokal getestet, bevor er auf dem Server konfiguriert wurde. Nach den erfolgreichen Tests überprüft der Scheduler stündlich den Intervall der Kunden und stellt sicher, dass der Import nur dann ausgeführt wird, wenn er tatsächlich geplant ist. Die Konfiguration wird in Anlage 14 aufgezeigt.

2.4. Kontrolle und Abschluss

2.4.1. Durchführen von Unit-Tests

Nach der Entwicklung des Plugins wurden Unit-Tests durchgeführt, um die Funktionsweise einzelner Komponenten unabhängig vom Gesamtsystem zu überprüfen. Ziel war es, sicherzustellen, dass die Kernfunktionen stabil und korrekt arbeiten. Als erstes wurde die Datenbankklasse DatabaseService getestet, um die grundlegenden Operationen wie das Einfügen, Aktualisieren und Abrufen von Datensätzen zu prüfen. Danach folgten Tests für die Importer- und Parser-Klassen, bei denen unter anderem geprüft wurde, ob externe JSON-Daten korrekt verarbeitet, strukturiert und in die Datenbank eingefügt werden. Die Tests wurden mit dem Framework PHPUnit (PHPUnit, 2025) durchgeführt. Dafür wurden gezielt Testklassen erstellt, die typische Anwendungsfälle (z. B. das Parsen eines Events, das Prüfen auf Duplikate) sowie Grenzfälle abbilden. Dabei kamen sogenannte Assertions zum Einsatz, mit denen die tatsächlichen Rückgabewerte mit den erwarteten verglichen wurden. Die Tests erfolgten zunächst lokal in einer isolierten Entwicklungsumgebung mit vorbereiteten Testdaten. Wo nötig, wurden Funktionen wie externe API-Aufrufe durch Mocks ersetzt, um den Test unabhängig von der Verfügbarkeit externer Clients durchführen zu können. Durch die strukturierte Vorgehensweise konnten Fehler erkannt und behoben werden. Die Unit-Tests bildeten damit eine wichtige Grundlage für die Qualitätssicherung sowie die nachfolgenden Funktions- und Systemtests. Ein Auszug befindet sich in Anlage 15.

2.4.2. Funktionstest und Systemtests

Nach den Unit-Tests wurde die vollständige Funktionalität des Plugins in der WordPress-Umgebung getestet. Dabei wurden alle Hauptfunktionen wie der Import, die Datenverarbeitung, die Duplikat Prüfung sowie der Export über die REST-API geprüft. Im Anschluss wurden zwei Systemtests durchgeführt: Der erste Test erfolgte in einer frischen WordPress-Installation, um sicherzustellen, dass das Plugin auch außerhalb der Entwicklungsumgebung ohne Fehler geladen und ausgeführt wird. Der zweite Test wurde auf dem Live-System durchgeführt, um die Einbindung in die bestehende Infrastruktur sowie die Datenverarbeitung unter realen Bedingungen zu überprüfen. Beide Tests verliefen erfolgreich: Das Plugin ließ sich problemlos installieren, konfigurieren und verwenden.

Importierte Events wurden korrekt übernommen und über die Adminoberfläche Quellen verwaltet. Auch der Export per API funktionierte wie vorgesehen und liefert Events im JSON-Format. Auszüge der Tests befinden sich in Anlage 16 Auszug von Testfällen, Anlage 17 Ergebnis importierter Events und Anlage 18 Rest-API Zugriff .

2.4.3. Eingehaltene Sicherheitsaspekte überprüfen

Da das Plugin nicht öffentlich zugänglich ist und der Zugriff auf das WordPress-Backend auf autorisierte Benutzer beschränkt wurde, war ein umfangreiches Sicherheitskonzept nicht erforderlich. Die Konfiguration und Nutzung des Plugins sind ausschließlich WordPress Administratoren vorbehalten, wodurch ein zusätzlicher Schutz vor unbefugtem Zugriff gewährleistet ist. Dennoch wurde das Prinzip „Security by Design“ beachtet. Besonders bei Datenbankoperationen kam die Verwendung von Prepared Statements zum Einsatz, um potenzielle SQL-Injection Angriffe bei einem kompromittierten System zu verhindern. Der Server ist ausschließlich über HTTPS erreichbar, weitere sicherheitsrelevante Server- Konfigurationen werden außerhalb des Projekts vorgenommen. Die REST-API des Plugins ist durch einen API-Key abgesichert, sodass nur berechtigte Kunden Zugriff auf exportierte Daten erhalten. Hier kann bei Bedarf auch ein Rate Limit ergänzt werden, um Missbrauch oder Überlastung zu verhindern. Da keine personenbezogenen Daten verarbeitet werden, waren keine zusätzlichen datenschutzrechtlichen Maßnahmen erforderlich.

2.4.4. Soll-Ist-Vergleich und Ausblick

Im Soll-Ist-Vergleich zeigt sich, dass die wesentlichen Anforderungen an das Projekt vollständig umgesetzt werden konnten. Das Plugin erfüllt die Zielvorgabe, Veranstaltungen automatisch aus einer externen JSON-Quelle in festgelegten Intervallen zu importieren, zu filtern, Dubletten zu vermeiden und zusätzlich eine Export-API bereitzustellen sowie Fehler zu vermerken. Die Events werden um Ihre Herkunft gekennzeichnet. Die entwickelte Lösung ist modular aufgebaut, erweiterbar und wurde ohne Änderungen am WordPress- oder MEC-Core umgesetzt. Die zentrale Anforderung an Wartbarkeit und Flexibilität wurde durch die gewählte Architektur mit Factory- und Strategy-Pattern erfüllt. Auch die geplante Integration eines Cronjobs für die automatische Ausführung konnte erfolgreich umgesetzt werden. Kleinere Abweichungen in der Zeitplanung, insbesondere im Bereich des Frontends, wurden im Projektverlauf erkannt und entsprechend angepasst, ohne das Endergebnis negativ zu beeinflussen. Für die Zukunft bietet das Plugin die Möglichkeit, weitere Importformate (z. B. XML,CSV oder iCal) sowie neue Datenquellen problemlos zu integrieren.

Ebenso denkbar ist eine Umstellung auf einen asynchronen Importprozess, um insbesondere bei großen Datenmengen oder mehreren parallelen Quellen eine bessere Performance und Nutzererfahrung zu ermöglichen. Ein solcher Ansatz wäre technisch realisierbar, z. B. über AJAX oder externe Hintergrundprozesse. Insgesamt wurde eine erweiterbare und zukunftssichere Lösung geschaffen, die den Pflegeaufwand erheblich reduziert und die Datenqualität langfristig verbessert.

2.4.5. Benutzungsanleitung und Verlaufsdokumentation

Während der Projektumsetzung wurde eine stichpunktartige Verlaufsdokumentation erstellt, um die einzelnen Schritte und Entscheidungen auch für den Betrieb nachvollziehbar festzuhalten. Zusätzlich wurde eine Benutzungsanleitung verfasst, die die Funktionen des Plugins im WordPress-Backend erklärt, insbesondere die Verwaltung von Datenquellen, Export-Kunden und den Import-Intervall. Die Anleitung wurde dem Plugin-Ordner beigefügt und steht für Nutzer und Entwickler zur Verfügung. In Anlage 19 Auszug Benutzerhandbuch und Anlage 20 Auszug Verlaufsdokumentation befinden sich entsprechende Auszüge.

2.4.6. Abnahme und Lesson learned

Zum Ende hin folgte die Abnahme durch den Projektbegleiter und danach an den Kunden. Es konnte das Ergebnis auf dem Server anhand der Anforderungen und Akzeptanzkriterien präsentiert werden und mit dem Benutzerdokument die Abnahme erfolgreich stattfinden. Während der Projektdurchführung zeigte sich, dass die ursprünglich geplante Zeitaufteilung angepasst werden musste. Insbesondere das Frontend erwies sich als aufwendiger als erwartet, da die Umsetzung der Admin-Oberflächen mehr Zeit in Anspruch nahm und zusätzliche Tests erforderlich waren. Auch wurde deutlich, dass in einzelnen Phasen mehr Rücksprache mit dem Projektbegleiter notwendig war als zunächst angenommen. Rückblickend wäre es sinnvoll gewesen, bestimmte Anforderungen und Details im Vorfeld noch klarer und umfassender abzustimmen. Insgesamt konnte ich durch das Projekt wertvolle Erfahrungen im Projektmanagement und in der realistischen Planung von Entwicklungsaufwänden sammeln.

2.4.7. Zeitkontrolle

Hier konnten nun die Zeiten verglichen werden und die Stunden eingetragen werden, diese in Anlage 5 zu finden sind. Es traten nur geringe Abweichungen auf, die den Gesamtzeitplan nicht beeinträchtigten und sich an anderer Stelle durch großzügig eingeplante Zeiten ausglichen.

3. Anlagen

Anlage 1 Auszug aus dem Pflichtenheft - Anforderungen

6. Anforderungen

6.1 Funktionale Anforderungen

1. Automatisierter Import externer Veranstaltungen

Der Importer soll Veranstaltungen aus Datenquellen automatisiert in den Modern Events Calendar (MEC) integrieren. Umgesetzt wird eine Quelle bzw. REST-API mit JSON-Format. Die Konfiguration der Datenquellen erfolgt dynamisch, sodass mehrere Quellen abgefragt werden können.

Die Importfunktion prüft, ob ein Event bereits existiert (über eine eindeutige externe ID) und entfernt den Bestand oder legt neue Events an.

2. Admin-Oberfläche

Das Plugin stellt eine benutzerfreundliche Admin-Oberfläche im WordPress-Backend bereit. Über diese können folgende Funktionen genutzt werden:

- Konfiguration des automatischen Import-Intervalls (z. B. 3× täglich)

- Verwaltung der Datenquellen (URLs, API-Keys)

- Konfiguration der Export-Clients (API-Key)

3. Zeitgesteuerte Synchronisierung

Ein geplanter Server-Cronjob führt den Importvorgang automatisch nach Angabe täglich aus. Die Häufigkeit ist konfigurierbar über die WordPress-Admin-Oberfläche (via Einstellungs-seite).

4. Dublettenvermeidung und Event-Kennzeichnung

Zur Vermeidung von Duplikaten und zur transparenten Datenpflege wird jedes Event mit einer zusätzlichen Information versehen.
In der Datenbank wird hierzu ein Feld event_origin verwendet, mit folgendem Wert

- import – für importierte Events

Diese Information kann für die Dubletten Prüfung sowie für spätere Filterungen oder Exportvorgänge verwendet werden.

Löschlogik:
Wird ein Event in der externen Quelle gelöscht, so wird es auch in MEC entsprechend entfernt oder deaktiviert.

5. Export-Schnittstelle für Partner

Ein REST-basierter JSON-Export-Endpoint über WordPress stellt eine filterbare Eventliste bereit, die Drittanbieter zur Weiterverarbeitung verwenden können.
Folgende Abfrageparameter werden unterstützt:

- since – liefert nur Events, seit einem bestimmten Zeitpunkt
- category– mögliche Filterung nach Event-Kategorien

6.2 Nicht-funktionale Anforderungen

1. Sicherheit

Sowohl der Import- als auch der Export-Endpunkt sind durch eine API-Key-basierte Authentifizierung gesichert. Der Schlüssel kann über die WordPress-Admin-Oberflächen verwaltet werden. Notwendige Sicherheitsaspekte des Systems werden bei der Entwicklung beachtet.

2. Fehlerbehandlung und Logging

Fehler beim Datenabruf oder bei der Verarbeitung werden protokolliert. Das Logging erfolgt:

- in eine Logdatei im Plugin-Verzeichnis
- optional in WordPress oder WP-Config

Fehlerhafte oder unvollständige Daten werden nicht übernommen. Das System kennzeichnet problematische Events zur manuellen Nachbearbeitung.

3. Kompatibilität und Wartbarkeit

Die Lösung muss vollständig MEC-Core-kompatibel sein und darf keine Änderungen am bestehenden Plugin-Code vornehmen.
Die Umsetzung erfolgt in einem eigenständigen WordPress-Plugin, das updatesicher konzipiert ist.

4. Schnittstellen-Spezifikation

Alle Kommunikationsschnittstellen basieren auf REST-GET-Anfragen und liefern Daten im JSON-Format. Import- und Exportprozesse verwenden HTTP-Kommunikation mit definierten Endpunkten und erlauben Statuscodes zur Fehlerbehandlung (z. B. 200 OK, 401 Unauthorized, 500 Internal Server Error).

Anlage 2 JSON-Beispiel der externen API

JSON-Beispiel


```
[{
    "event_id": "183665883",
    "hauptkategorie_id": "Familie",
    "hauptkategorie": "Familie",
    "unterkategorie_id": "",
    "unterkategorie": "",
    "veranstaltungsort": {
        "id": "737403",
        "name": "Rathaus",
        "gemeinde_id": "Wertheim",
        "gemeinde": "Wertheim",
        "ortsteil_id": "",
        "ortsteil": "",
        "strasse": "M\u00fchlenstra\u00dfe",
        "plz": "97877",
        "telefon": "",
        "telefax": "",
        "mobile": "",
        "email": "",
        "url": "",
        "lat": "49.757603",
        "lng": "9.517751",
        "koordx": "0",
        "koordy": "0",
        "koordz": "0"
    },
    "veranstalter": {
        "id": "737801",
        "name": "AG Jugendhilfeplanung",
        "gemeinde_id": "",
        "ortsteil_id": "",
        "telefon": "",
        "telefax": "",
        "mobile": "",
        "email": "",
        "url": "",
        "html": "",
        "lat": "0.000000",
        "lng": "0.000000",
        "koordx": "0",
        "koordy": "0",
        "koordz": "0"
    },
    "titel": "Elternkompass: Das Geheimnis gl\u00fccklicher Kinder",
    "kurztext": "<p>Eine Veranstaltung f\u00fcr Eltern von Kindern von ein
bis zu 10 Jahren<\/p>  <p><span style=\"text-decoration:
underline;\">Referent<\/span>: Ernst Hartmann, Diplom-Psychologe an der
Erziehungs- und Familienberatungsstelle des Caritasverbandes im Tauberkreis
e.V.<\/p>",
    "langtext": "<p>Erziehung ist oft eine gar nicht so einfache Sache und
schon gar nicht kann man alle Situationen \u00fcber einen Kamm
scheren.<\/p>  <p>Ziel des Vortrags ist es Informationen zu vermitteln, wie
Worte und S\u00e4tze die kindliche Seele beeinflussen, welche Emotionen in
```


*** vom Kunden erhalten**

```
Kindern angelegt sind und diese f\u00fcr die Zukunft weiterentwickelt
werden k\u00f6nnen. Es ist wichtig f\u00fcr die Kinder da zu sein, aber
auch wichtig sich selbst als Mutter oder Vater nicht zu verleugnen und die
eigenen W\u00fcnsche angemessen zu ber\u00fccksichtigen.<\/p>  <p><span
style=\"text-decoration: underline;\">Mindestteilnehmerzahl<\/span>:
5<br><span style=\"text-decoration: underline;\">Anmeldung<\/span>: Bis 3
Werktage vor der Veranstaltung unter Telefon 09342\/301-310 oder <a
href=\"mailto:uwe.schloer-kempf@wertheim.de\">uwe.schloer-
kempf@wertheim.de<\/a>.<\/p>",
   "datum_zeit": {
       "datum" : "2020-10-05",
       "zeit" : [ {
          "von" : "09:00",
          "bis" : "12:00",
          "text" : "f\u00fcr geladene G\u00e4ste"
       }, {
          "von" : "14:00",
          "bis" : "17:00"
       } ]
    },
   "multimedia": [{
      "url": "http:\/\/url.de\/bild.jpg",
      "kategorie": "bild",
      "titel": "",
      "copyright": "",
      "fotograf": "",
      "agentur": "",
      "ort": ""
   }]
},
```


Anlage 3 Risikoanalyse - Auszug aus dem Pflichtenheft

Risiko	**Eintritt**	**Auswirkung**	**Risikominderung**
Datenquellen-Strukturänderung	Mittel	Hoch	Erweiterbare und Wartbare implementierung für flexible Parser-Anpassung
SQL-Injection / DB-Sicherheit	Gering	Mittel	Prepared Statements
Automatisierungsfehler (Import)	Mittel	Mittel	Logging in DB & Logdatei
Duplettenbildung	Hoch	Mittel	Duplikatprüfung implementiert
Cronjob-Ausfall	Gering	Hoch	Server-Cron statt WP-Cron für Stabilität

Anlage 4 Wirtschaftlichkeitsberechnung

Vorgang - Ausgaben	Zeit	Kosten/Stunde	Kosten
Entwicklung	80 h	€	€
Rückmeldung/Rücksprachen	4 h	€	€
Abnahme	1 h	€	€
		Gesamt	€
Vorgang - Einnahme	**Satz**	**Kosten**	**Gesamt**
Grundfunktion	1	€ netto	€
+ Quellenanbindung	1	€ netto	€
		Gesamt	€

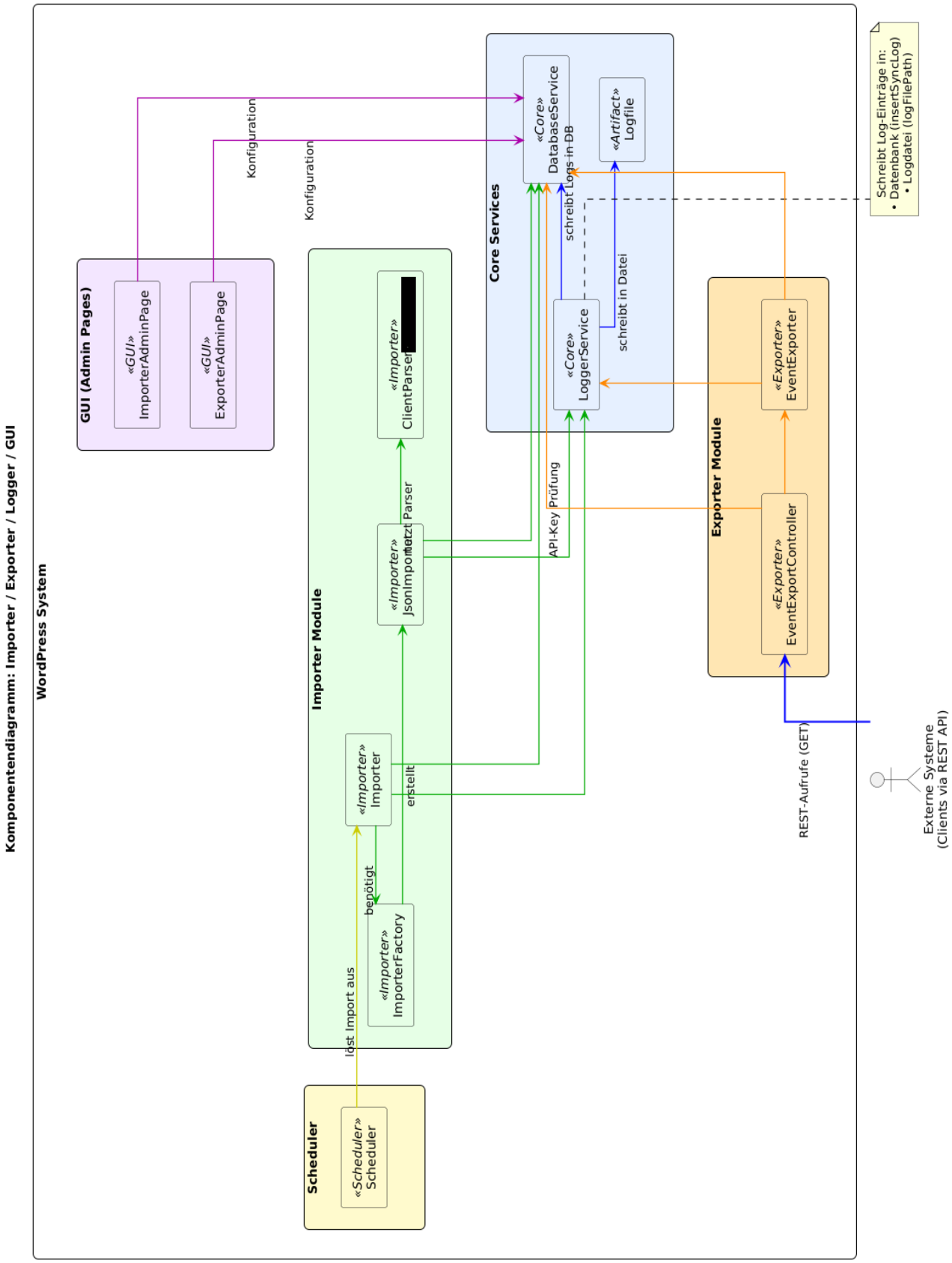
$$Einnahmen - Kostenaufwand = Gewinn$$

$$X - Y = Z \text{ € } netto$$

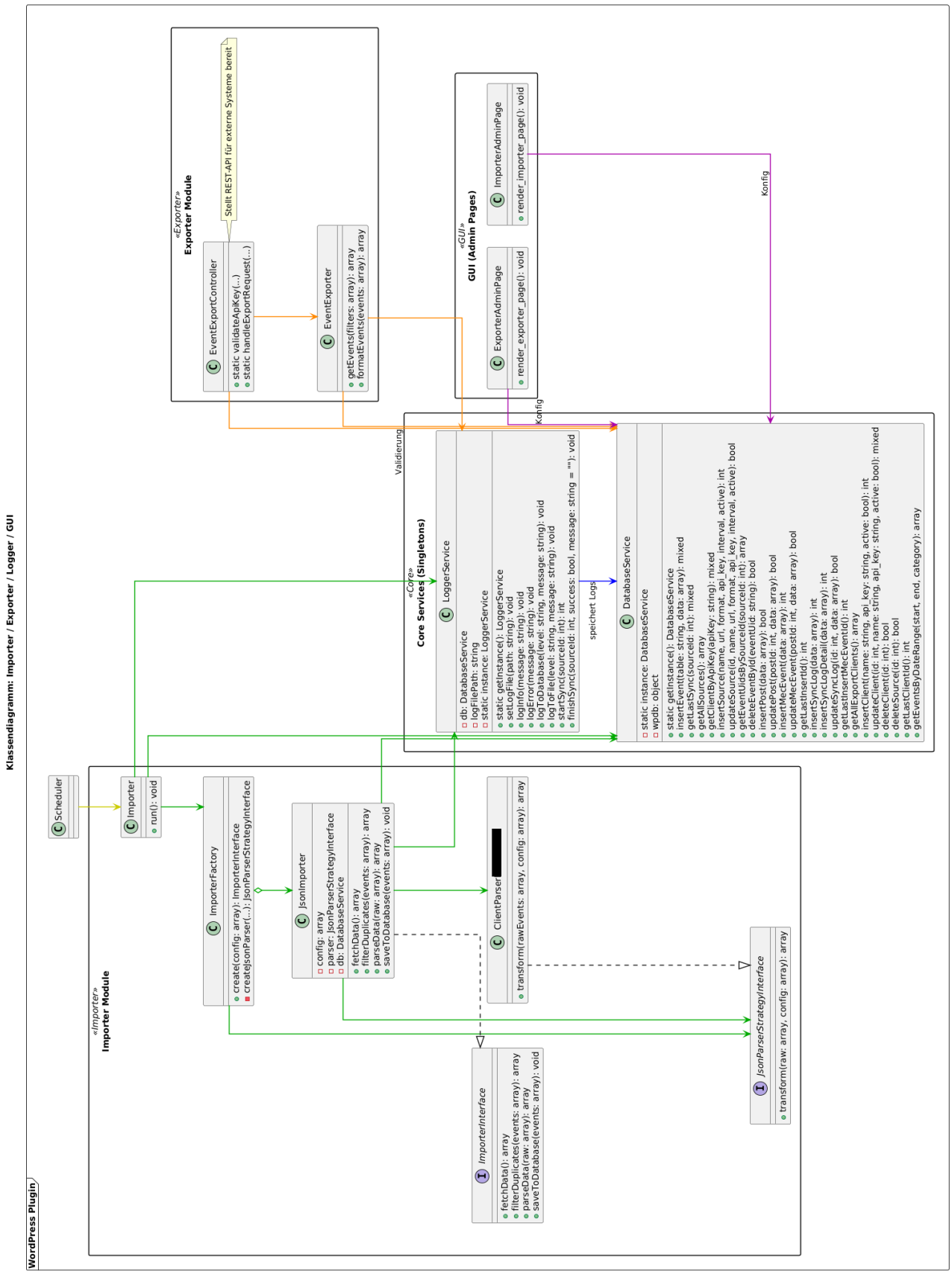
Anlage 5 Zeitplan Soll-Ist-Vergleich

	Soll	Ist	DIF
2.1. Information	**5h**	**5h**	**0h**
2.1.2. Klärung der externen Datenquelle/Formate	0,5h	0,5h	
2.1.3. Ist-Analyse	0,5h	0,5h	
2.1.4. Soll-Konzept und Rahmenbedingungen	1h	1h	
2.1.5. Analyse der externen und internen APIs	1h	1h	
2.1.6. Risikobewertung	1h	1h	
2.1.7. Wirtschaftlichkeitsanalyse	1h	1h	
2.2. Planung	**14h**	**11h**	**+3h**
2.2.1. Arbeitsschritte und Zeitplan anpassen	1h	1h	
2.2.2. Alternativen und Machbarkeit	1,5h	1,5h	
2.2.3. Arbeitsmittel	0,5h	0,5h	
2.2.4. Erstellung des Pflichtenhefts	2h	2h	
2.2.5. Entwurf der Plugin Struktur und des Lösungskonzept	4h	2h	+2h
2.2.6. Entwurf des Datenbankmodells	2h	2h	
2.2.7. Entwurf der Admin-Oberflächen	3h	2h	+1h
2.3. Durchführung	**38h**	**41h**	**-3h**
2.3.1. Datenbank Konzept umsetzen	6h	2h	+4h
2.3.2. Entwicklung der Admin-Oberflächen	6h	16h	-10h
2.3.3. Entwicklung des Import Moduls	12h	12h	
2.3.4. Entwicklung des Export Moduls	10h	8h	+2h
2.3.5. Konfiguration des Schedulers	4h	3h	+1h
2.4. Kontrolle und Abschluss	**23h**	**23h**	**0h**
2.4.1. Durchführung von Unit-Tests	4h	4h	
2.4.2. Funktionstests und Systemtests	5h	5h	
2.4.3. Eingehaltene Sicherheitsaspekte überprüfen	1h	1h	
2.4.4. Soll-Ist-Vergleich und Ausblick	2h	2h	
2.4.5. Benutzungsanleitung und Verlaufsdokumentation	1h	1h	
2.4.6. Abnahme und Lesson learned	1h	1h	
2.4.7. Zeitkontrolle	1h	1h	
2.4.8. Erstellung der Projektdokumentation	8h	8h	
Gesamt	**80h**	**80h**	**0h**

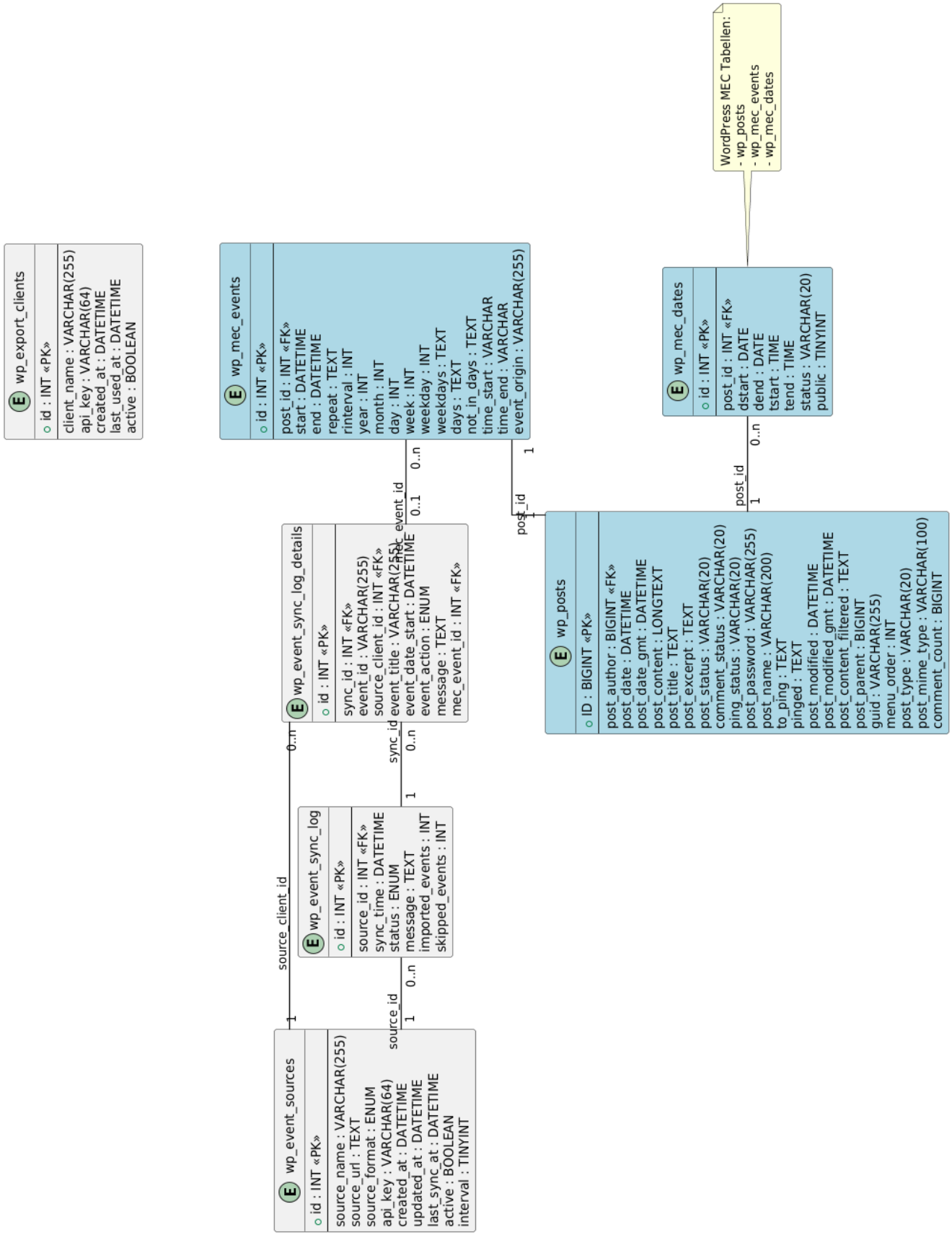
Anlage 6 Komponentendiagramm



Anlage 7 Klassendiagramm

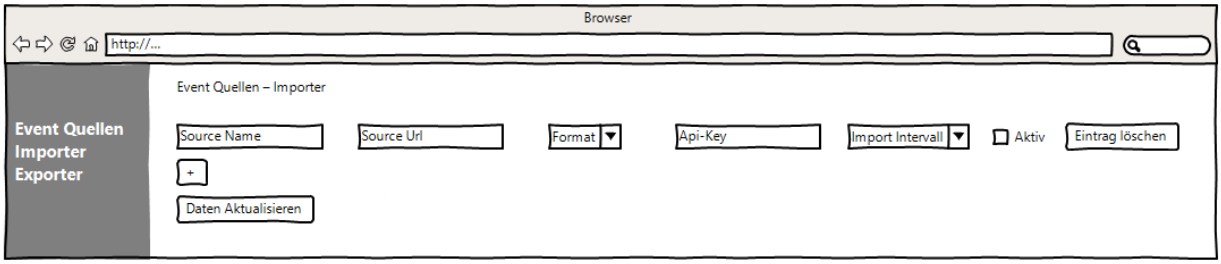


Anlage 8 Datenbanktabellen – ER-Modell

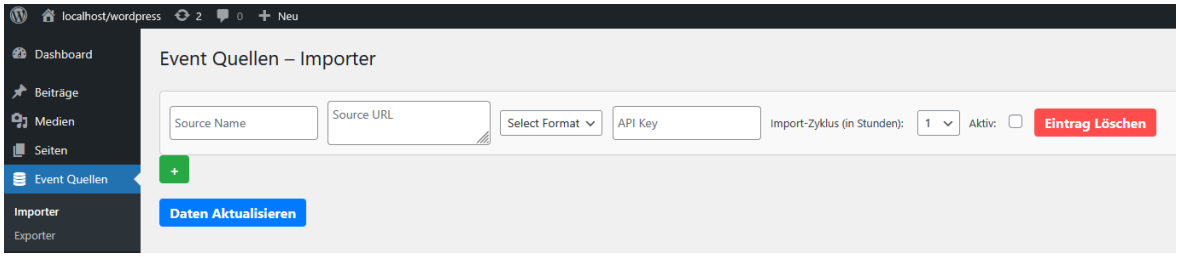


Anlage 9 Wireframes und GUIs

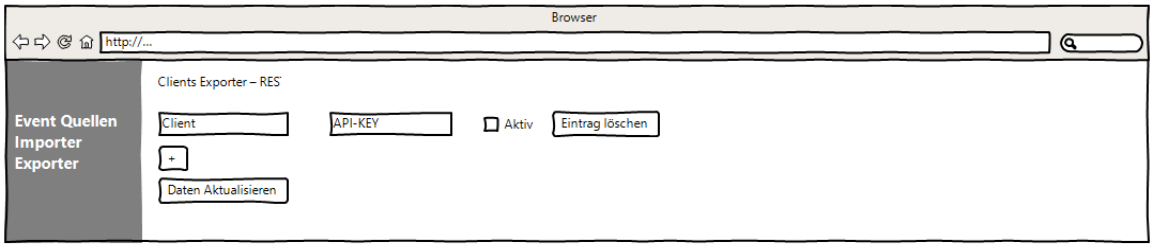
Quellen Importer:



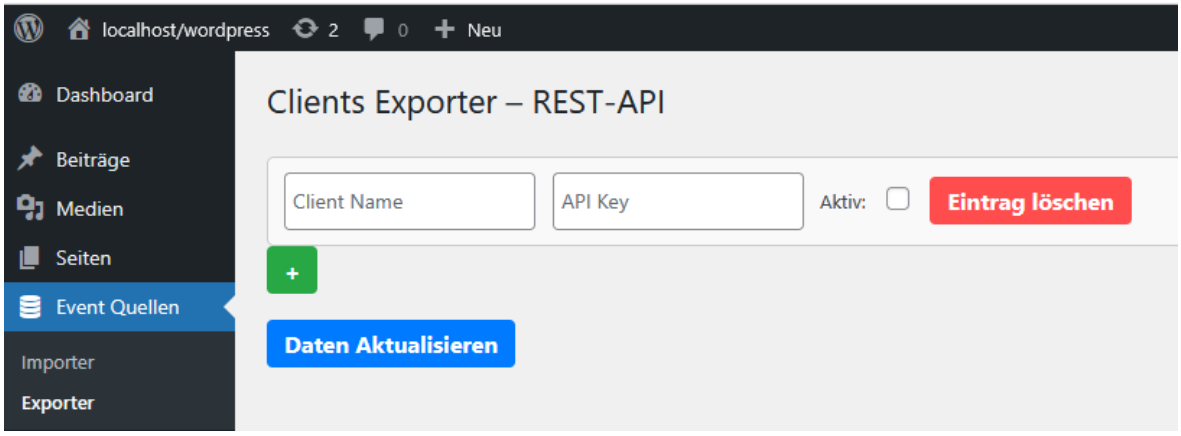
Ergebnis:



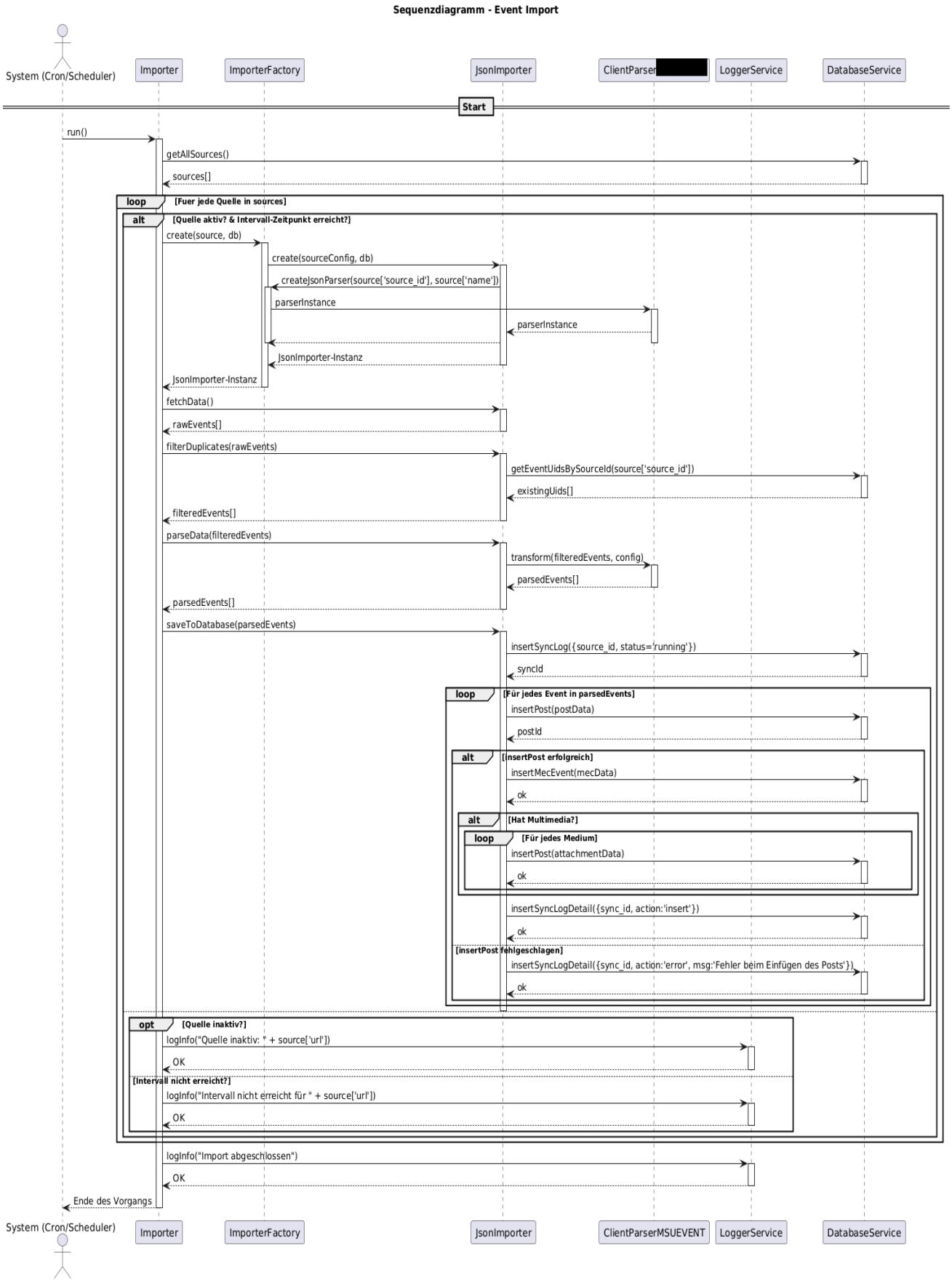
Kunden Exporter:



Ergebnis:



Anlage 10 Sequenzdiagramm Importer



Anlage 11 Code Auszug Importer und ImporterFactory Klassen

```php
class Importer
{

    public function run(): void
    {

        $logger = LoggerService::getInstance();
        $db = DatabaseService::getInstance();


        try {
            $sources = $db->getAllSources();
        } catch (Exception $e) {
            $logger->logInfo("Fehler beim Abrufen der Quellen: " . $e->getMessage());
            exit;
        }

        foreach ($sources as $source) {
            if ($source['active'] == 1) {
                $interval = $source['interval'];
                $lastSync = strtotime($source['last_sync_at']);
                $now = time();

                if ($lastSync === false) {
                    $lastSync = 0;
                }

                if ($now - $lastSync >= $interval * 60) {
                    $importer = ImporterFactory::create($source, $db);

                    $raw = $importer->fetchData();
                    $filteredEvents = $importer->filterDuplicates($raw);
                    $parsed = $importer->parseData($filteredEvents);
                    $importer->saveToDatabase($parsed);
                } else {
                    $logger->logInfo("Intervall nicht erreicht für " . $source['url'])
                }
            } else {
                $logger->logInfo("Quelle inaktiv: " . $source['url']);
            }
        }

        $logger->logInfo("Import abgeschlossen");
    }
}
```

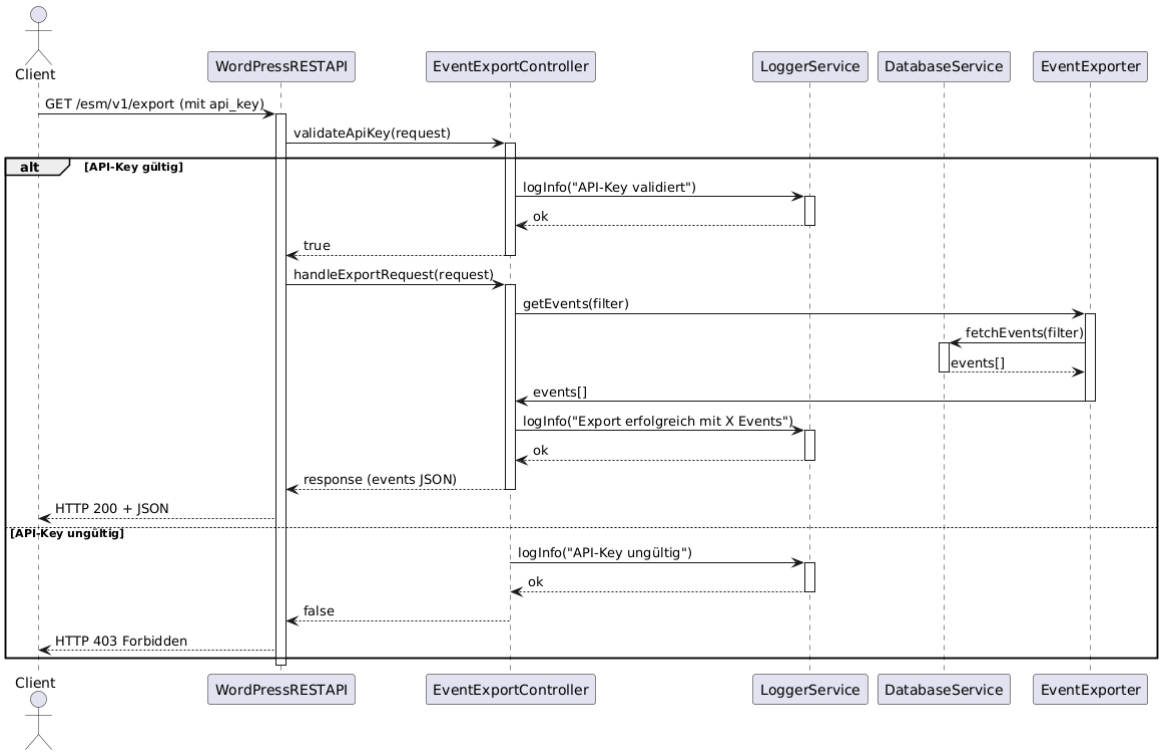
```php
class ImporterFactory
{
    public static function create(array $config, DatabaseService $db): ?ImporterInterface
    {
        $logger = LoggerService::getInstance();

        switch (strtolower($config['source_format'])) {
            case 'json':
                return new JsonImporter($config, self::createJsonParser($config['id'], $config['source_name']), $db);
                // weitere Formate
            default:
                $logger->logError("Unbekanntes Format: {$config['source_format']}");
                return null;
        }
    }

    private static function createJsonParser(int $sourceid, string $name): ?JsonParserStrategyInterface
    {
        $logger = LoggerService::getInstance();

        switch ($sourceid) {
            case 1:
                return new ClientParser[redacted]();
            default:
                $logger->logError("Unbekannter Parser: {$sourceid}, {$name}");
                return null;
        }
    }
}
```

Anlage 12 Sequenzdiagramm Exporter



Anlage 13 Code Auszug EventExportController Klasse

```php
class EventExportController
{

    public static function validateApiKey(WP_REST_Request $request): bool
    {
        $logger = LoggerService::getInstance();

        try {
            $apiKey = $request->get_param('api_key');

            if (!$apiKey) {
                $logger->logInfo('API-Key fehlt in der Anfrage');
                return false;
            }

            $db = DatabaseService::getInstance();
            $client = $db->getClientByApiKey($apiKey);
            //Client aktiv gesetzt?
            if ($client && $client->active) {
                $logger->logInfo("API-Key validiert: Client-ID {$client->id} ({$client->client_name})");
                return true;
            }

            $logger->logInfo('Ungültiger API-Key oder inaktiver Client');
            return false;
        } catch (Exception $e) {
            $logger->logInfo('Fehler bei API-Key-Validierung: ' . $e->getMessage());
            return false;
        }
    }

    public static function handleExportRequest(WP_REST_Request $request)
    {
        $logger = LoggerService::getInstance();

        $filters = [
            'from'     => $request->get_param('from'),
            'to'       => $request->get_param('to'),
            'category' => $request->get_param('category')
        ];

        try {
            $exporter = new EventExporter();
            $events = $exporter->getEvents(array_filter($filters));

            $logger->logInfo("Export erfolgreich mit " . count($events) . " Events");
            return rest_ensure_response($events);
        } catch (Exception $e) {
            $logger->logInfo("Fehler beim Export: " . $e->getMessage());
            return rest_ensure_response(['error' => 'Export fehlgeschlagen'], 500);
        }
    }
}
```

Anlage 14 Server-Cronjob

```
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command

# Cronjob fuer den Importer
0 * * * * /usr/bin/php /var/www/html/wp-content/plugins/event-source-manager/Importer/Importer.php > /dev/null 2>&1

root@f3dc1ba08989:/var/www/html#
```

Anlage 15 Auszug Unit Tests

```php
    public function testGetSingleton()
    {
        $db = DatabaseService::getInstance();
        $actualObject = DatabaseService::getInstance();
        $this->assertSame($db, $actualObject);
    }

    public function testImporterandDoubleIdsInDatabase()
    {
        //Set Up
        $db = DatabaseService::getInstance();

        // Quelle in der DB mit json Format
        $sources = $db->getAllSources();

        $jsonimporter = ImporterFactory::create($sources[0], $db);
        $this->assertInstanceOf(JsonImporter::class, $jsonimporter);


        $raw = $jsonimporter->fetchData();
        $filteredEvents = $jsonimporter->filterDuplicates($raw);
        $parsed = $jsonimporter->parseData($filteredEvents);
        // $parsed muss leer sein da die Events aus dem Mockup
        // bereits in der Datenbank vorhanden sind
        $this->assertEmpty($parsed);
    }
}
```

```
Time: 00:00.758, Memory: 56.00 MB

OK (2 tests, 3 assertions)
```

{"version":1,"defects":[],"times":{"MyTestClass::testGetSingleton":0.41,"MyTestClass::testImporterandDoubleIdsInDatabase":0.343}}

Unit Tests

Methode / Einheit	Eingangsbedingung	Erwartetes Ergebnis	Ergebnis
DatabaseService::getInstance()	Zweimal instanziiert	Gleiche Instanz	Ok
Duplikatprüfung	Liste enthält doppelte IDs	werden herausgefiltert	Ok
ImporterFactory->create()	format = JSON	Gibt korrekte Json-Importer-Klasse zurück	Ok

Anlage 16 Auszug von Testfällen

Systemtests

Funktion / Anforderung	Eingangsbedingung	Test-Schritte	Erwartetes Ergebnis	Ergebnis
Automatischer Import neuer Events	Quelle aktiv, Cronjob ausgelöst	Cronjob starten, MEC-DB prüfen	Neue Events werden korrekt importiert	Ok
Duplikatprüfung	Event-ID bereits vorhanden	Importprozess erneut auslösen	Duplikate werden ignoriert	Ok
API-Erreichbarkeit (negativ)	JSON-API nicht erreichbar	Importprozess starten	Fehler wird korrekt geloggt	Ok
Export-REST-API mit gültigem Key	Client mit API-Key aktiv	API-Request an Endpunkt senden	JSON-Daten werden zurückgegeben	Ok
Admin-UI Quellenverwaltung	Backend geladen	Daten Eingabe	Quelle erscheint in wp_event_sources	Ok
JsonParser->transform()	Daten aus der Quelle geholt	JsonParser und transform() Methode	Gibt korrekt strukturiertes Array zurück	Ok

Anlage 17 Ergebnis importierter Events

	id	sync_id	event_id	source_client_id	event_title	event_date_start	event_action	message	mec_event_id
Bearbeiten Kopieren Löschen	10	2	183665883	1	Elternkompass: Das Geheimnis glücklicher Kinder	2025-10-21 09:00:00			14
Bearbeiten Kopieren Löschen	11	2	183665884	1	Konzertabend mit regionalen Künstlern	2025-11-15 18:00:00			15
Bearbeiten Kopieren Löschen	12	2	183665885	1	Bewegung macht Spaß – Kinderturnen für alle	2025-12-01 16:00:00			16
Bearbeiten Kopieren Löschen	13	2	183665886	1	Workshop Kreatives Schreiben	2025-12-10 10:00:00			17
Bearbeiten Kopieren Löschen	14	2	183665887	1	Yoga für Anfänger	2025-12-12 17:00:00			18

	id	source_id	sync_time	status	message	imported_events	skipped_events
Bearbeiten Kopieren Löschen	2	1	2025-10-22 16:57:10		Import abgeschlossen	5	0

Alle auswählen *markierte:* Bearbeiten Kopieren Löschen Exportieren

Anlage 18 Rest-API Zugriff

Erlaubt mit API-Key:

Datei Bearbeiten Ansicht Chronik Lesezeichen Extras Hilfe

localhost/wordpress/wp-json/e:

http://localhost/wordpress/wp-json/esm/v1/export?api_key=sk-1234abcd5678efgh9012ijkl3456mnop

JSON Rohdaten Kopfzeilen

Speichern Kopieren Alle einklappen Alle ausklappen JSON durchsuchen

```
0:  { event_id: "14", event_start: "2025-10-21 09:00:00", event_end: "2025-10-21 12:00:00", … }
1:  { event_id: "15", event_start: "2025-11-15 18:00:00", event_end: "2025-11-15 21:00:00", … }
2:  { event_id: "16", event_start: "2025-12-01 16:00:00", event_end: "2025-12-01 17:30:00", … }
3:  { event_id: "17", event_start: "2025-12-10 10:00:00", event_end: "2025-12-10 13:00:00", … }
4:  { event_id: "18", event_start: "2025-12-12 17:00:00", event_end: "2025-12-12 18:30:00", … }
```

Verweigert falscher API-Key:

Datei Bearbeiten Ansicht Chronik Lesezeichen Extras Hilfe

localhost/wordpress/wp-json/e:

localhost/wordpress/wp-json/esm/v1/export?api_key=123456789

JSON Rohdaten Kopfzeilen

Speichern Kopieren Alle einklappen Alle ausklappen JSON durchsuchen

```
code:      "rest_forbidden"
message:   "Du bist leider nicht berechtigt, diese Aktion durchzuführen."
data:
   status: 401
```

Anlage 19 Auszug Benutzerhandbuch

3. Admin-Oberfläche – Importer

Der Importer dient der Verwaltung externer Datenquellen, aus denen Veranstaltungsdaten automatisch in das MEC-System importiert werden. Jede Quelle repräsentiert eine externe Plattform oder Daten-API, die Events bereitstellt.

Folgende Eingabefelder stehen zur Verfügung:

• **Source Name:** Bezeichnung der Datenquelle, dient zur internen Identifikation.

• **Source URL:** Vollständige URL der API-Endpunktadresse, von der Eventdaten abgerufen werden.

• **Source Format:** Format der importierten Daten (JSON, XML, CSV). Aktuell wird JSON empfohlen.

• **API Key:** Authentifizierungsschlüssel für geschützte Quellen für die Zugangsberechtigung.

• **Import-Zyklus (in Stunden):** Intervall, in dem die Daten automatisch synchronisiert werden.

• **Aktiv:** Schaltet die Quelle aktiv oder inaktiv. Nur aktive Quellen werden importiert.

Anlage 20 Auszug Verlaufsdokumentation

2.3.4 Entwicklung des Export-Moduls

- REST-API-Routen in der Haupt-Plugin-Datei registriert
- Sicherheit und Authentifizierung über callback und permission_callback umgesetzt
- Instanziierung der EventExportController-Klasse mit Methoden validateApiKey() und handleExportRequest()
- validateApiKey() prüft eingehenden API-Key auf Existenz und Gültigkeit über DatabaseService
- Bei erfolgreicher Validierung Aufruf von handleExport() → Übergabe an EventExporter Datenbank Events auslesen

2.4.2 Funktionstest und Systemtests

- Vollständige Funktionalität (Import, Export, Duplikaterkennung) erfolgreich getestet
- Tests in neuer WordPress-Installation sowie im Live-System durchgeführt
- Erfolgreiche Installation, Aktivierung und Nutzung des Plugins in beiden Umgebungen
- Automatischer Cron-Import überprüft (korrekter Intervall und fehlerfreie Ausführung)
- Duplikatprüfung anhand Event-ID getestet
- Export über REST-API getestet (mit und ohne gültigen API-Key)
- API-Responses in JSON-Format überprüft
- Logging und Fehlerbehandlung abgefragt

Anlage 21 Glossar

Begriff	Kurzbeschreibung
API-Key	Authentifizierungsschlüssel für den Zugriff auf eine API.
Cronjob	Zeitgesteuerter, automatischer Prozess auf dem Server.
CSS	Sprache zur Gestaltung des Layouts einer Website.
Factory Pattern	Entwurfsmuster zur Erzeugung passender Klasseninstanzen.
HTTP	Übertragungsprotokoll für die Kommunikation im Internet.
JSON	Datenformat (JavaScript Object Notation).
Lesson learned	Gewonnene Erkenntnisse und Erfahrungen
MEC	Wordpress Plugin welches Event Daten verwaltet
Mock	Platzhalterobjekt, das reale Komponenten im Test simuliert.
PHPUnit	Framework zur Durchführung automatisierter Tests in PHP.
Prepared Statement	Sichere SQL-Abfrage zur Vermeidung von Injection-Angriffen.
REST-API	Schnittstelle für Webkommunikation über HTTP.
Security by Design	Konzept, bei dem Sicherheit von Anfang an integriert wird.
Singleton Pattern	Muster, das nur eine Instanz einer Klasse zulässt.
SQL	Sprache zur Verwaltung und Abfrage von Datenbanken.
Strategy Pattern	Muster zur flexiblen Auswahl von Algorithmen.
Unit-Test	Test einzelner Software-Komponenten auf korrekte Funktion.
Wireframe	Schematische Darstellung einer Benutzeroberfläche.
XAMPP	Cross, Apache, MySQL, MariaDB, PHP und Perl

4. Quellen

- WireframeSketcher: Produktseite, https://wireframesketcher.com/ , abgerufen am 2.10.2025
- WordPress.org: Dokumentation, https://wordpress.org/documentation/ , abgerufen am 15.10.2025
- WordPress.org: Plugins, https://de.wordpress.org/plugins/ , abgerufen am 21.09.2025
- Bergmann, Sebastian: PHPUnit – The PHP Testing Framework.
URL: https://phpunit.de , abgerufen am 15.10.2025